

- Alex Beatson and Ryan P Adams. Efficient optimization of loops and limits with randomized telescoping sums. In *International Conference on Machine Learning*, 2019.
- James Bergstra, Olivier Breuleux, Frédéric Bastien, Pascal Lamblin, Razvan Pascanu, Guillaume Desjardins, Joseph Turian, David Warde-Farley, and Yoshua Bengio. Theano: a CPU and GPU math expression compiler. In *Proceedings of the Python for Scientific Computing Conference (SciPy)*, volume 4, 2010.
- Christian Bischof, Alan Carle, George Corliss, Andreas Griewank, and Paul Hovland. ADIFOR—generating derivative codes from Fortran programs. *Scientific Programming*, 1(1):11–29, 1992.
- Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 2018.
- Tian Qi Chen, Jens Behrmann, David K Duvenaud, and Jörn-Henrik Jacobsen. Residual flows for invertible generative modeling. In *Advances in Neural Information Processing Systems*, pp. 9913–9923, 2019.
- Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016.
- Krzysztof M Choromanski and Vikas Sindhwani. On blackbox backpropagation and Jacobian sensing. In *Advances in Neural Information Processing Systems*, pp. 6521–6529, 2017.
- John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(Jul):2121–2159, 2011.
- Conal Elliott. The simple essence of automatic differentiation. *Proceedings of the ACM on Programming Languages*, 2(ICFP):70, 2018.
- Aidan N Gomez, Mengye Ren, Raquel Urtasun, and Roger B Grosse. The reversible residual network: Backpropagation without storing activations. In *Advances in Neural Information Processing Systems*, pp. 2214–2224, 2017.
- A Griewank and U Naumann. Accumulating Jacobians by vertex, edge, or face elimination. *cari* 2002. In *Proceedings of the 6th African Conference on Research in Computer Science, INRIA, France*, pp. 375–383, 2002.
- Andreas Griewank and Andrea Walther. Algorithm 799: revolve: an implementation of checkpointing for the reverse or adjoint mode of computational differentiation. *ACM Transactions on Mathematical Software (TOMS)*, 26(1):19–45, 2000.
- Andreas Griewank and Andrea Walther. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, volume 105. SIAM, 2008.
- Laurent Hascoet and Valérie Pascual. The Tapenade automatic differentiation tool: Principles, model, and specification. *ACM Transactions on Mathematical Software (TOMS)*, 39(3):20, 2013.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778, 2016.
- Geoffrey E Hinton. Training products of experts by minimizing contrastive divergence. *Neural computation*, 14(8):1771–1800, 2002.
- Sepp Hochreiter and Jürgen Schmidhuber. Flat minima. *Neural Computation*, 9(1):1–42, 1997.
- Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. In *International Conference on Learning Representations*, 2017.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

- Diederik P Kingma and Max Welling. Auto-encoding variational Bayes. *arXiv preprint arXiv:1312.6114*, 2013.
- Quoc V Le, Navdeep Jaitly, and Geoffrey E Hinton. A simple way to initialize recurrent networks of rectified linear units. *arXiv preprint arXiv:1504.00941*, 2015.
- Yucen Luo, Alex Beatson, Mohammad Norouzi, Jun Zhu, David Duvenaud, Ryan P Adams, and Ricky TQ Chen. Sumo: Unbiased estimation of log marginal probability for latent variable models. In *International Conference on Learning Representations*, 2020.
- Dougal Maclaurin, David Duvenaud, and Ryan P Adams. Autograd: Effortless gradients in numpy. URL <https://github.com/HIPS/autograd>.
- Dougal Maclaurin, David Duvenaud, and Ryan Adams. Gradient-based hyperparameter optimization through reversible learning. In *International Conference on Machine Learning*, pp. 2113–2122, 2015.
- Ryan G. McClarren. *Computational Nuclear Engineering and Radiological Science Using Python: Chapter 18 - One-Group Diffusion Equation*. Academic Press, 2018.
- Uwe Naumann. Optimal accumulation of Jacobian matrices by elimination methods on the dual computational graph. *Mathematical Programming*, 99(3):399–421, 2004.
- Uwe Naumann. Optimal Jacobian accumulation is NP-complete. *Mathematical Programming*, 112(2):427–441, 2008.
- Samuel L Smith and Quoc V Le. A Bayesian perspective on generalization and stochastic gradient descent. In *International Conference on Learning Representations*, 2018.
- Xu Sun, Xuancheng Ren, Shuming Ma, and Houfeng Wang. meprop: Sparsified back propagation for accelerated deep learning with reduced overfitting. In *Proceedings of the 34th International Conference on Machine Learning*, pp. 3299–3308. JMLR. org, 2017.
- Corentin Tallec and Yann Ollivier. Unbiasing truncated backpropagation through time. *arXiv preprint arXiv:1705.08209*, 2017.
- Bart van Merriënboer, Dan Moldovan, and Alexander Wiltchko. Tangent: Automatic differentiation using source-code transformation for dynamically typed array programming. In *Advances in Neural Information Processing Systems*, pp. 6256–6265, 2018.
- Andrea Walther and Andreas Griewank. Getting started with ADOL-C. *Combinatorial Scientific Computing*, (09061):181–202, 2009.
- Bingzhen Wei, Xu Sun, Xuancheng Ren, and Jingjing Xu. Minimal effort back propagation for convolutional neural networks. *arXiv preprint arXiv:1709.05804*, 2017.
- Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3-4):229–256, 1992.

APPENDIX A: NEURAL NETWORK EXPERIMENTS

RANDOM PROJECTIONS FOR RAD

As mentioned in Section 3.2 (around Equation 5) of the main paper, we could also use different matrices P that have the properties

$$\mathbb{E}P = I_d \quad \text{and} \quad P = RR^T, R \in \mathbb{R}^{d \times k}, k < d.$$

In the appendix we report experiments of letting R be a matrix of iid Rademacher random variables, scaled by $\sqrt{k}$. $P = RR^T$ defined in this way satisfies the properties above. Note that this would lead to additional computation: The Jacobian or input vector would have to be fully computed, and then multiplied by R and stored. In the backward pass, it would have to be multiplied by R^T . We report results as the “project” experiment in the full training/test curves in the following sections. We see that it performs competitively with reducing the mini-batch size.

ARCHITECTURES USED

We use three different neural network architectures for our experiments: one fully connected feedforward, one convolutional feedforward, and one recurrent.

Our fully-connected architecture consists of:

1. Input: 784-dimensional flattened MNIST Image
2. Linear layer with 300 neurons (+ bias) (+ ReLU)
3. Linear layer with 300 neurons (+ bias) (+ ReLU)
4. Linear layer with 300 neurons (+ bias) (+ ReLU)
5. Linear layer with 10 neurons (+ bias) (+ softmax)

Our convolutional architecture consists of:

1. Input: $3 \times 32 \times 32$ -dimensional CIFAR-10 Image
2. 5×5 convolutional layer with 16 feature maps (+ 2 zero-padding) (+ bias) (+ ReLU)
3. 5×5 convolutional layer with 32 feature maps (+ 2 zero-padding) (+ bias) (+ ReLU)
4. 2×2 average pool 2-d
5. 5×5 convolutional layer with 32 feature maps (+ 2 zero-padding) (+ bias) (+ ReLU)
6. 5×5 convolutional layer with 32 feature maps (+ 2 zero-padding) (+ bias) (+ ReLU)
7. 2×2 average pool 2-d (+ flatten)
8. Linear layer with 10 neurons (+ bias) (+ softmax)

Our recurrent architecture was taken from Le et al. (2015) and consists of:

1. Input: A sequence of length 784 of 1-dimensional pixels values of a flattened MNIST image.
2. A single RNN cell of the form

$$h_t = \text{ReLU}(W_{ih}x_t + b_{ih} + W_{hh}h_{t-1} + b_{hh})$$

where the hidden state (h_t) dimension is 100 and x_t is the 1-dimensional input.

3. An output linear layer with 10 neurons (+ bias) (+ softmax) that has as input the last hidden state.

CALCULATION OF MEMORY SAVED FROM RAD

For the baseline models, we assume inputs to the linear layers and convolutional layers are stored in 32-bits per dimensions. The ReLU derivatives are then recalculated on the backward pass.

For the RAD models, we assume inputs are sampled or projected to 0.1 of their size (rounded up) and stored in 32-bits per dimension. Since ReLU derivatives can not exactly be calculated now, we